Take-away Sheets

Applied AI and Automation for Executives. One page for each session: what you keep.

Session 1

Seeing: pixels, features and neurons

How can a machine read a picture when all it has is numbers?

- SEEING: pixels, features and neurons
- A grey picture is a grid of numbers: 8 x 8 = 64. A colour pixel is 3 numbers (red, green, blue): 8 x 8 x 3 = 192.
- A feature is a measurement from the input, such as the floor area of a house or the ink in the centre of a digit.
- Weighted sum: multiply each input by its weight and add. z = w1x1 + w2x2 + b. House price = 2 x floor area + 10 x bedrooms + 30 (the bias, the base price), in UGX million.
- Neuron: weighted sum + bias, then an activation function. Umbrella: 0.1 x rain + 0.05 x minutes - 6 = 2.5; a step gives 1, carry it. a = f(z). ReLU(z) = max(0, z). Sigmoid: 1/(1 + e^(-z)).
- Without an activation function, stacked layers collapse into one weighted sum. A layer is a table: 16 neurons are 16 rows of weights, a = f(Wx + b).
- One weight learns by the loop: predict, measure the loss, take a gradient descent step: w new = w old - eta x gradient (eta is the learning rate). Loss L = 1/2 (y - prediction)^2; for many examples, MSE = (1/n) sum of squared errors. Gradient here = -x(y - prediction) = −4, so 0.5 - 0.1 x (−4) = 0.9, then 1.14, 1.284. Backpropagation is the chain rule: dL/dw = dL/dprediction x dprediction/dz x dz/dw.
- A network stacks layers: 64 inputs, a hidden layer of 16 neurons, 10 outputs. 1,210 parameters (weights and biases).
- An epoch is one full pass over the training data. In the recorded run, 17% right after 1 epoch and 97% after 200.
- Recorded run, 5 November 2026: 436 of 450 unseen digits right. Shift each digit by 1 pixel: 46% right.
- Ask of any image system: what was it trained on, what was it tested on, and what does a person check?

I keep: A one-page sheet: how a picture becomes numbers, a neuron worked by hand on a house price and an umbrella, and the digit results.

Session 2

Learning from examples: types of machine learning, decision trees, nearest neighbours

How does a machine learn a rule from examples, and how much can I trust what it learned?

- LEARNING FROM EXAMPLES
- Four types: supervised (examples with labels), unsupervised (finds groups), reinforcement (trial and reward), self-supervised (predicts a hidden part of its own data).
- Classification predicts a category (spam or not spam). Regression predicts a number (a house price).
- A decision tree is yes-or-no questions: splits and leaves. It picks the split that makes the cleanest groups (Gini impurity G = 1 - sum of p squared: 0.44 before, 0.29 after the sky question; information gain = 0.44 - 0.29 = 0.16; entropy H = -sum of p log2 p is the alternative). It reads as plain rules you can audit.
- Overfitting: a tree grown too deep fits its training data, noise included: 100% on training claims, 88% on new, against 93% for a small tree.
- Distance metric: Euclidean d = square root of the sum of (a - b)^2 (3 across and 4 up is 5) or Manhattan d = sum of |a - b| (3 + 4 = 7). k-nearest neighbours predicts from the k most similar examples: average for a number (250, 270, 190 gives 237), majority vote for a category. Put features on one scale first, min-max scaling x′ = (x - min)/(max - min): 69% raw, 90% scaled.
- Words must become numbers first. A first step is a bag of words: counts of words in each message. Session 4 builds this properly: TF-IDF, then embeddings.
- Cross-validation (k-fold): split into k folds, train on k-1, test on the held-out fold, rotate. Score = mean of the k fold scores; report the average and the spread. For a number, the error is the mean squared error from Session 1.
- Recorded run, 5 November 2026: on 40 customer messages the learned models scored about 55% (range of single tests very wide). A simple keyword rule got 23 of 40. Forty examples is too few for text.
- The expense claims are fictional. The 40 messages are Nile Valley Foods practice files.
- Ask of any model: how many examples, how was it tested, and who checks the doubtful cases?

I keep: A one-page sheet: the four types of learning, a tree read as rules, k-nearest neighbours, and the honest scores from today's run.

Session 3

Measuring a model: accuracy, precision, recall, F1, ROC and AUC, thresholds, baselines, overfitting

How do I know whether a model is any good, and what should I ask the vendor who says it is?

- VENDOR QUESTION SHEET: THE NUMBERS BEHIND AN AI CLAIM
- Words: confusion matrix (TP, FP, FN, TN). Precision = TP ÷ (TP + FP). Recall (true positive rate) = TP ÷ (TP + FN). Accuracy = (TP + TN) ÷ (TP + TN + FP + FN). False positive rate = FP ÷ (FP + TN). F1 = 2 × precision × recall ÷ (precision + recall), the harmonic mean.
- ROC curve: true positive rate plotted against false positive rate for every threshold. AUC = the area under it (0.5 coin flip, 1 perfect). Cost of errors = (TP + FP) × cost of chasing a flag + FN × cost of a miss. Smoke alarm check: accuracy 96.2%, precision 32%, recall 80%, false positive rate 3.5%, F1 46%.
- 1. What test data was the figure measured on, and had the model seen that data?
- 2. How common is the thing we look for? What does the majority-class baseline score?
- 3. What are precision and recall, and at which threshold?
- 4. Who chose the threshold, and what did each kind of mistake cost?
- 5. How does it compare with a simple rule or with our current process?
- 6. How many real cases were in the test? Few cases means a shaky figure.
- 7. Can we test it on 50 of our own cases before we sign?
- 8. How will you check it after launch, as our data changes?
- 9. Who reads the false positives, and how long does it take them?
- 10. What happens to a case the model gets wrong? Who can overrule it?
- Remember: accuracy rewards doing nothing when the thing is rare. A perfect score on training data can mean overfitting.

I keep: The vendor question sheet: ten questions about the numbers behind any AI claim. Keep it next to every proposal you read.

Session 4

Words as numbers: bag of words, TF-IDF, embeddings and distance

How can a computer find the right clause or policy when I describe it in plain words?

- CHEAT SHEET: THE FOUR MEASURES OF HOW CLOSE TWO VECTORS ARE
- Symbols: a = (a_1, a_2) and b = (b_1, b_2) are two vectors. a_1 and b_1 are their first numbers (dimension 1). $\|a\|$ is the length of a, $\sqrt{(a_1^2 + a_2^2)}$. θ (theta) is the angle between them.
- EUCLIDEAN DISTANCE: $d(a, b) = \sqrt{[(a_1 - b_1)^2 + (a_2 - b_2)^2]}$. The straight-line gap. Smaller is closer. Includes length.
- MANHATTAN DISTANCE: $d(a, b) = |a_1 - b_1| + |a_2 - b_2|$. The city-block gap: add the gaps. Smaller is closer.
- DOT PRODUCT: $a \cdot b = a_1b_1 + a_2b_2$. Multiply matching numbers, add. Bigger is closer. Long vectors score higher.
- COSINE SIMILARITY: $\cos(\theta) = (a \cdot b) \div (\|a\| \times \|b\|)$. The angle only. Bigger is closer, 1 is the same direction. Ignores length.
- Worked example, three friends rate two films out of 10: Ana a = (3, 4), Ben b = (6, 8), Chloe c = (4, 3). Euclidean Ana to Ben 5, to Chloe 1.4. Manhattan 7 and 2. Dot product 50 and 24. Cosine 1.00 and 0.96.
- Any number of dimensions: Euclidean = $\sqrt{\Sigma} (a_i - b_i)^2$, industry name L2 distance. Manhattan = $\Sigma |a_i - b_i|$, L1 distance. Dot product = Σa_ib_i, the inner product. $\|a\| = \sqrt{\Sigma} a_i^2$. Cosine distance = 1 − cosine similarity. Σ means add up over every dimension i.
- TF-IDF as used in the recorded run (scikit-learn defaults): $idf(t) = \ln[(1 + N) \div (1 + df(t))] + 1$, weight = tf × idf, then divide by the length of the message vector. Textbook form: $tf \times \log(N \div df)$. N is the number of messages, df the number containing the term.
- The build-up: bag of words counts, TF-IDF weights rare terms, an embedding is a vector that represents meaning, and the four measures say how close two vectors are.
- Remember: a score only ranks. Read the top three. Semantic search finds descriptions, exact search finds codes.
- Ask the vendor: which measure, and were the vectors set to length 1 first?

I keep: The cheat sheet of the four measures: Euclidean distance, Manhattan distance, dot product and cosine similarity, each with its formula, its picture and when to use it.

Session 5

Language models: tokens, attention and next-word probability

How does a language model turn your words into an answer, and why can the same question give different answers?

- Language model on a page
- Training: the Session 0 loop on text. Hide the next token, predict, measure the loss, adjust the weights by gradient descent. The text is its own label (self-supervised).
- Token and tokenizer: text is cut into tokens from a fixed vocabulary (50,257 in GPT-2). The context window is how many tokens fit at once (1,024 in GPT-2).
- Next-token prediction: the model gives a logit to every token in the vocabulary, softmax turns logits into a probability distribution, one token is sampled, added, and the loop repeats.
- Softmax: p_i = e^(z_i) / sum_j e^(z_j), in words probability = 2.72 to the logit, divided by the total of all of them. The probabilities add up to 100 percent. With temperature: p_i = e^(z_i/T) / sum_j e^(z_j/T).
- Loss and objective: next-token loss (cross-entropy) L = -ln p(correct token), for example -ln 0.18 = 1.71. The probability of a text is P(x1...xn) = product over t of P(x_t given the earlier tokens); training lowers the average loss.
- Temperature: divide the logits by T first. Low repeats, high wanders. Greedy decoding takes the top token. Top-k keeps the k most probable tokens. Top-p keeps the smallest group adding up to p. Kept probabilities are divided by their total.
- Self-attention: each token makes a query (what I look for), a key (what I offer) and a value (what I pass on). Score = query x key, softmax gives weights, output = weights x values: Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V (the toy uses one number each, so d_k = 1). One attention head shows one pattern.
- Transformer block: self-attention plus a feed-forward network. Embedding matrix: a lookup table with one row of numbers per token. Positional encoding gives each token its position: x_t = E[id_t] + P[t]. GPT-2 small stacks 12 blocks.
- Five questions for any answer: Where did this come from? Can I check it against a source? Would it change if I asked again? Does it contain figures or quotes? Who reviews it?
- Recorded runs: GPT-2 small, 124M parameters, 6 November 2026.

I keep: A one-page sheet: next-token prediction, how the model is trained, the softmax formula, what temperature, top-k and top-p do, self-attention with query, key and value, and five questions to ask about any language-model answer.

Session 6

Foundation models: pretraining, post-training, model families, reasoning and multimodal models, benchmarks

What is a foundation model, how does it become an assistant, and what does a benchmark score tell me?

- Model comparison grid
- Terms: pretraining (next-token prediction on a huge text corpus) gives a base model. Post-training (supervised fine-tuning, also called instruction tuning, then RLHF, reinforcement learning from human feedback) gives an instruction-tuned model. A foundation model is a large pretrained model many products build on; an LLM is a large language model.
- Formulas: pretraining loss L = -(1/N) sum of ln P(x_t given the earlier tokens), the average next-token cross-entropy over the corpus. Perplexity = e^L (lower is better). Benchmark accuracy = correct / total. Mixture of experts: output = sum over chosen experts of gate weight g_i x expert output E_i.
- Families: mixture of experts (a router picks 2 of 8 experts per token), reasoning model (writes steps first), multimodal model (images and sound too), diffusion model (removes noise step by step), small language model (laptop size).
- My task (one line): _____
- Models to try (2 or 3): _____
- Ten real cases with the answer I expect: attach a list
- Three checks (for example figures, label, nothing invented): _____
- Model card questions: licence? size (parameters)? trained on? limits? what is missing?
- Benchmark claim questions: which test (MMLU, GSM8K, HumanEval, Chatbot Arena)? what setup? what date? how does it match my task?
- Recorded runs: 6 November 2026. Qwen2.5-1.5B-Instruct and GPT-2 run locally; Claude in a fresh subagent session. Not ChatGPT or Gemini.

I keep: A one-page model comparison grid for your own work, with the questions to ask of a model card and a benchmark claim.

Session 7

Open weights, file formats, size and running cost

What does it mean to own, host or rent a model?

- RENT, HOST OR OWN: DECISION SHEET
- Task: _____ Data it touches: _____ Sensitive? (yes or no)
- Memory sum: memory ≈ N × b (N parameters, b bytes each: FP32 4, FP16 2, INT8 1, 4-bit 0.5). Example: 7 billion × 2 bytes (FP16) = 14 GB; 7 billion × 0.5 byte (4-bit) = 3.5 GB
- Quantisation (affine): q = round(x ÷ s) + z, read back $\hat{x}$ = s (q − z). Break-even volume = F ÷ (p − v): fixed cost a month ÷ (price per use − running cost per use).
- Open source (OSI definition) or open weights? Which licence? Which uses does it limit?
- File format (safetensors, GGUF, ONNX)? Precision (FP32, FP16, INT8, 4-bit)? Quality tested on my cases?
- Route: rent / host / own / wait. Reason: _____
- One thing that would change my mind: _____

I keep: A one-page rent, host or own decision sheet, with the memory sum and the licence questions to ask.

Session 8

Prompting and reasoning models

How do I get reliable work out of an assistant?

- PROMPT TEMPLATE AND LOG
- Five parts: Role. Context. Examples. Format. Constraints (and what to say when unsure).
- My prompt: _____
- Evaluation set: test cases with known answers (three to ten): _____
- Log row: Version ___ What changed ___ Runs ___ How many right ___ Accuracy = correct ÷ total ___ Next change ___
- Temperature (Session 5): p = e^(z/T) ÷ Σ e^(z/T). Few-shot: the model predicts P(output | instructions, examples, input).
- Who checks the figures before use: _____
- Template: Templates/Reusable Brief Template.docx (seven parts)

I keep: A reusable prompt template and a prompt log, so that every prompt you keep has been tested on an evaluation set.

Session 9

Giving a model your knowledge: retrieval-augmented generation, long context and fine-tuning

How do I give an AI model my company's knowledge, and when is each way worth it?

- PROMPTING, RAG OR FINE-TUNING: ask in this order.
- 1. Does a clearer prompt with the page pasted in give right answers on ten checked questions? Yes: stop.
- 2. Are the facts too many, too long or changing? Use retrieval-augmented generation (RAG). Check your chunks. Let the model say not stated.
- 3. Is the style or format still wrong after steps 1 and 2? Consider fine-tuning, with examples you own and personal data removed.
- Always: measure on the same ten questions before you add the next method.
- Formulas: cosine similarity $\cos\theta = q \cdot c / (|q||c|)$, equal to $q \cdot c$ when both lengths are 1 (0.73 for clause 3.2, 0.47 for 3.1). Top-k retrieval keeps the k chunks with the largest scores. Prompt budget: prompt + retrieved text + answer ≤ context window (Q03: 94 + 104 + 300 = 498 tokens; whole handbook: 94 + 1,074 + 300 = 1,468). LoRA: $W' = W + B\,A$, with B (d × r) and A (r × k), so $r(d + k) = 8 \times (768 + 2{,}304) = 24{,}576$ trainable numbers per block instead of $d \times k = 1{,}769{,}472$; 12 blocks give 294,912.
- Terms: chunking, embedding, vector database, retriever, top-k retrieval, context window, LoRA (low-rank adaptation), adapter, rank.
- Remember: a high similarity score shows that topics are close. It does not show that the answer is present. Fine-tuning shapes style and does not reliably teach facts.
- Worksheet: six NVF requests. (a) Summarise one supplier email. (b) Answer staff questions from a handbook that changes each quarter. (c) Every customer reply ends with the house sign-off. (d) Check a price against one pasted quote. (e) Answer questions about 400 contracts. (f) Draft replies in the Sales Manager voice from 200 past emails.

I keep: A one-page decision tree: prompting first, then retrieval-augmented generation (RAG), then fine-tuning, and the test that tells you when to move on.

Session 10

When models fail, and how to check them

How do I find out when an AI answer is wrong, before it costs me?

- FIVE-STEP CHECK. The more an error would cost, the more steps you take.
- 1. Source: ask for the quote or clause, then open it.
- 2. Recount: redo every number by hand or with a formula.
- 3. Compare: check the key, or a second source.
- 4. Run twice: if the same request gives different answers, look closer.
- 5. Sign: a named person decides and records it.
- EVALUATION-SET TEMPLATE. Id | Question | Ground truth | Source | Run 1 | Run 2 | Cost of a wrong answer. Ten questions, two with no answer, kept apart from anything used to tune the tool.
- TERMS. Hallucination: confident output the facts do not support. Prompt injection: text in the input that tries to steer the model. Bias: results that favour or miss some groups. Data leakage: test answers or private data reaching where they should not. Overfitting: fits the data it was tuned on and does worse on new data. Baseline: a trivial method to beat.
- FORMULAS. Accuracy = correct ÷ total; the baseline of always saying not stated scores 2 ÷ 10 = 20 percent, and three clauses score 10 ÷ 10 = 100 percent. Expected cost of an error = P(error) × cost of an error (illustration: 2 ÷ 12 ≈ 0.17 × UGX 1,000,000 ≈ UGX 167,000, with the cost invented).
- Score right, missed, invented. Compare with the baseline of always saying not stated. Run again after any change.
- Recorded finding: most runs were right and the faults were small. Only checking found them.

I keep: A five-step check routine and an evaluation-set template (questions with ground truth), each on one page.

Session 11

Agentic AI: tools, loops, MCP, autonomy and guardrails

What changes when an assistant can act as well as answer?

- Tool Boundary Sheet (the permissions table): fill it before any agent is switched on.
- Agent: an AI model with a goal and tools. The agent loop: plan, act, observe, repeat; it stops when the goal is met, a step limit is reached or a person is needed.
- Tool use (function calling): the model writes a request; a program runs the tool and returns the result.
- Permissions (scopes) and least privilege: grant the smallest set that does the job (granted $\subseteq$ needed), like a house-sitter with the front door key only. Read and draft are enough for a mail summary.
- Risk = likelihood × impact. Each extra permission raises the impact of a fooled agent; send makes it permanent.
- Four rows: Read, Write, Send, Spend. For each: it may, a person approves, it must never.
- Human-in-the-loop approval: sends, payments, deletions and commitments always need a named person. Approval time $T = n \times t$: 3 approvals × 2 minutes = 6 minutes in the demo mailbox.
- Guardrails: permissions, approval, an audit log line for every step, one owner, stop conditions, an off switch that works in minutes.
- Prompt injection: addressed to the AI, asks for data to leave or a deletion, asks for silence. Stop and tell the owner.
- Model Context Protocol (MCP): a standard plug (host, one client per server; servers offer tools, resources, prompts). It connects; it does not decide permissions.
- Autonomy levels: 0 answers, 1 suggests, 2 drafts and you approve, 3 acts then reports, 4 acts alone. Choose the lowest that works.

I keep: Your Tool Boundary Sheet, the permissions table for an agent: for read, write, send and spend, what it may do, what a person approves (human-in-the-loop) and what it must never do, with the owner, the audit log and the off switch.

Session 12

Workflow mapping (practical)

Which of my tasks should be a rule, an AI step or an agent?

- Workflow: a task that repeats. Map it: trigger, steps, handoffs (red), check (gold), one lane per person or system. Everyday example: making tea.
- Automation: a machine runs the steps when the trigger fires. Rule-based automation follows fixed rules, such as a bank standing order.
- Decide each step in order: clear words or numbers: rule-based automation. Meaning with a quick check: AI step. Several tools and its own next step: agent with written permissions. Money, trust or law: a person (human-in-the-loop).
- Side by side on twelve customer messages: rule 7 of 12; AI 9 exact, 11 acceptable, 1 wrong with confidence (M14). Different measures; small sample.
- Baseline: measure today first. Hours saved $H = n \times (t_0 - t_1) \div 60$: runs a month × minutes saved per run ÷ 60 (× people, if several do it). Review time is a cost.
- Net benefit = benefits − costs. Payback period = set-up cost ÷ net saving a month. ROI = net benefit ÷ cost × 100%. Kettle example: payback 60,000 ÷ 20,000 = 3 months, net benefit 12 × 20,000 − 60,000 = UGX 180,000, ROI 180,000 ÷ 60,000 × 100% = 300% (invented).
- NVF plan example: 4 × 80 ÷ 60 = 5.3 hours a month before review time (4 hours after 20 minutes of review a run); net benefit 304,000 − 145,000 = UGX 159,000 a month; payback period 480,000 ÷ 159,000 ≈ 3.0 months; 12-month net 12 × 159,000 − 480,000 = UGX 1,428,000; first-year ROI 1,428,000 ÷ 2,220,000 ≈ 64% (invented inputs).
- Assistant sketch: trigger, inputs, outputs, fewest permissions, who approves, the off switch.
- Measure four weeks: minutes each run with review, errors and their cost, cases sent to a person, your stop rule.
- Templates: Task Card, Use-Case Scorecard, Tool Boundary Sheet (permissions table), Business Case Workbook (Inputs sheet).

I keep: Your mapped workflow, the choice for each step, your assistant sketch, and the baseline and hours-saved estimate in the Business Case Workbook.

Session 13

Governance and buying (practical)

What rules must my company set for AI, and how do I question what I am sold?

- AI governance: the policies, roles and checks that direct and control AI use. Everyday twin: the rules for lending your car (policy, risk list, classification, ask first, logbook, check the garage).
- Acceptable use policy (the AI Use Policy template): eleven sections. I drafted 3 approved tools, 4 data classification rules, 5 human oversight, 7 accounts, 8 incidents, 10 review date.
- Data classification, four questions: living person (personal data)? harmful if leaked? staff only? already published? The strictest answer wins.
- Risk register: score R = L × I, likelihood (1 to 5) times impact (1 to 5), a risk matrix. High 15 or more, Medium 8 to 14, Low below 8. A control lowers likelihood or impact.
- Every limit of a model gives a rule: human oversight, approval on every send plus an audit log, data classification, licence read, least privilege, re-run the evaluation set after a change.
- Total cost of ownership (TCO) = set-up + months × monthly fee. OrbitDesk: 0 + 36 × USD 2,400 = USD 86,400 before usage.
- Vendor red flags: no number, no approval, trains on my data, no references, no cap, no exit, a deadline, bank login.
- Vendor due diligence, five steps: ask in writing, score 0 to 2, ask for proof, pilot with a stop rule, sign with an exit.
- Law, my understanding: the Data Protection and Privacy Act 2019 covers personal data (sections 19, 20, 21, 23, 29). Ask my lawyer.
- Frameworks: NIST AI RMF (voluntary), ISO/IEC 42 001 (an AI management system standard a vendor can be certified against), OECD AI Principles, African Union Continental AI Strategy, EU AI Act (only if I serve the EU).

I keep: Your Risk Register, your AI Use Policy draft, your Vendor Question List with five questions, and your page of questions for the lawyer.

Session 14

Data readiness assessment, roadmap and 90-day plan (practical)

How ready is my organisation's data for AI, what will I fix first, and who will own it?

- Everyday twin: a pantry audit. Data inventory (the list), metadata (the labels), lineage (where it came from), data governance (who may open the cupboard), quality dimensions (the checks), roadmap (the shopping plan).
- Six data quality dimensions: completeness, accuracy, consistency, timeliness, uniqueness, validity. Completeness = filled values ÷ all values × 100% (NVF PO numbers: 58 ÷ 60 = 96.7%). Uniqueness = distinct records ÷ all records (59 ÷ 60 = 0.983).
- Five kinds of data: tables (structured), text, images, audio, sensor data. First steps: define columns, find the current version, read images with OCR, transcribe audio, check gaps.
- Readiness scorecard: Inventory, Readable, Complete and unique, Metadata, Rights, Current. Green 2, Amber 1, Red 0. Readiness score = Σ points, out of 12 (NVF: 2+1+1+1+0+1 = 6). A Red on rights stops the work.
- Six steps: inventory, digitise, clean, describe, protect, index and test. Start with one folder and one task.
- First test: an evaluation set of 20 questions with known answers, two or three traps, a line for success written first. Chunking splits text into passages; an embedding is a vector for meaning.
- Every roadmap action: owner, start, due date, hours, depends on, a success measure (a KPI) someone else can check. Total effort = Σ hours (NVF: 4+6+8+5+10+2 = 35). KPI change = (new − old) ÷ old: score 6 to 9 is +50%.
- Check-in milestones: day 30 (controls in place? gap 1 closed?), day 60 (stop rule met? actions on time?), day 90 (re-score, then continue, expand, pause or stop).
- Within 24 hours: book the three dates in my diary and tell each owner their action.
- The follow-up kit: 30-60-90 Check-in Form, Results Tracker, Sponsor Briefing.

I keep: Your inventory and scorecard, your roadmap with its risks and decisions, the Follow-up Kit (Check-in Form, Results Tracker, Sponsor Briefing) and your answer to the Session 0 question.